Module 6 - Containerized Applicant Analysis

Architecture, operations, verification, and Docker Hub images

GitHub source: pgozgo/jhu_software_concepts/module_6

Docker Hub web image: pgozgo/module6-web | Docker Hub worker image: pgozgo/module6-worker

Architecture

Browser localhost:8080	**Web** Flask :8080 non-root app	**RabbitMQ** tasks exchange tasks_q durable queue	**Worker** prefetch 1 non-root app	**PostgreSQL** applicants + analytics persistent volume

Service roles and ports

Service	Role / exposure
web	Flask dashboard and queue publisher; host 127.0.0.1:8080.
worker	Long-running task consumer; no host port. Reads applicant JSON from a read-only bind mount.
rabbitmq	Durable task broker; AMQP 5672 is private; management UI is bound to 127.0.0.1:15672.
db	PostgreSQL 16; container port 5432 is private; data persists in postgres_data.

Message and data flow

1. Pull Data enqueues **scrape_new_data**; Update Analysis enqueues **recompute_analytics**. Both endpoints return HTTP 202 promptly. Messages contain compact JSON with kind, UTC timestamp, and payload, and use persistent delivery mode 2.

2. Publisher and worker declare the same durable exchange, queue, and binding. The worker consumes with prefetch_count=1, validates JSON, and dispatches through its task-handler map.

3. Each valid delivery runs inside a PostgreSQL transaction. Scrapes read the Grad Cafe watermark, use the existing scraper for newer result IDs, insert applicants with URL conflict handling, advance the watermark, and refresh analytics. The inserts and watermark commit together. Analytics-only tasks refresh materialized views.

4. The worker acknowledges only after a successful commit. Handler/database failures roll back and are rejected with requeue=false; malformed tasks are rejected safely. Reload the dashboard to see refreshed values.

Initialization and local run

On a fresh PostgreSQL volume, the official entrypoint runs **src/db/init.sql** to create roles, applicants, and ingestion_watermarks with least-privilege grants. After PostgreSQL is healthy, worker startup runs **python -m db.load_data --file /app/src/data/applicant_data.json** before consuming tasks. The loader normalizes JSON, loads only when applicants is empty, skips duplicate URLs, seeds the watermark, and refreshes analytics. PostgreSQL data persists in the named postgres_data volume.

Install Docker Desktop and start its engine. From the repository root, use PowerShell: `Set-Location .\module_6`, `Copy-Item .env.example .env`, edit .env with distinct URL-safe passwords, then run `docker compose up --build`. Dashboard: http://localhost:8080. RabbitMQ management: http://localhost:15672. The management login uses RABBITMQ_USER and RABBITMQ_PASSWORD from the local .env file.

Published Docker Hub images

The public v1 tags were verified with Docker manifest inspection: `pgozgo/module6-web:v1` and `pgozgo/module6-worker:v1`. Repository pages: module6-web and module6-worker.

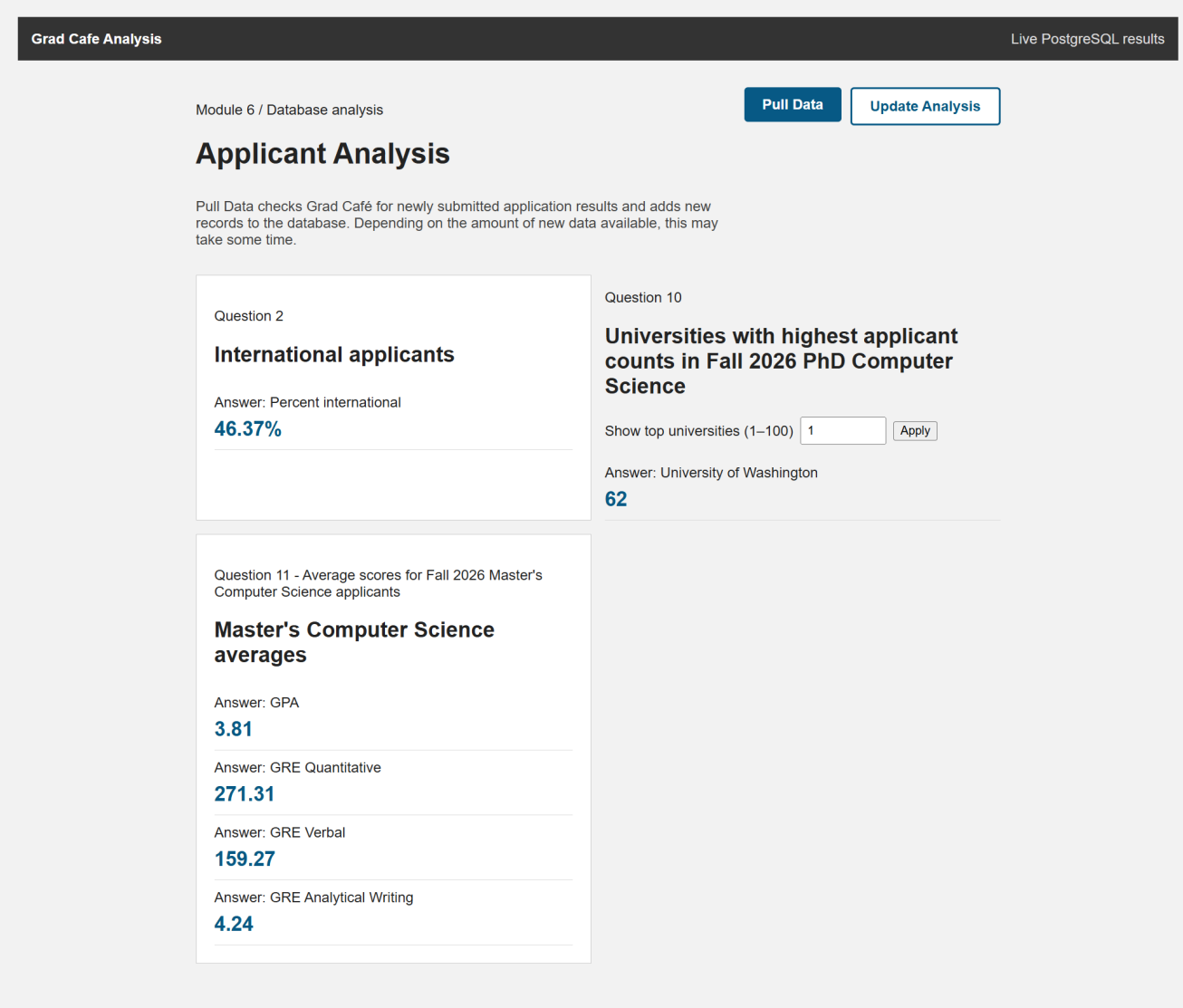
With .env configured, pull and run the published images without rebuilding: `docker pull pgozgo/module6-web:v1`, `docker pull pgozgo/module6-worker:v1`, `docker compose pull web worker`, then `docker compose up -d --no-build`. The repository workflow can publish latest, v1, and commit-SHA tags; configure GitHub Actions secrets DOCKERHUB_USERNAME and DOCKERHUB_TOKEN before running **Publish Module 6 images to Docker Hub**.

Verification

Local checks: 33 unit tests passed; 100% statement coverage for Flask app/publisher, task topology, worker consumer, and database loader; Pylint 10.00/10; Compose configuration validated; web and worker images built and ran as UID 10001. Both Docker Hub v1 manifests are available. The live dashboard and RabbitMQ management UI returned HTTP 200; all four Compose services were healthy. Coverage intentionally excludes the separate HTML/network scraper module.

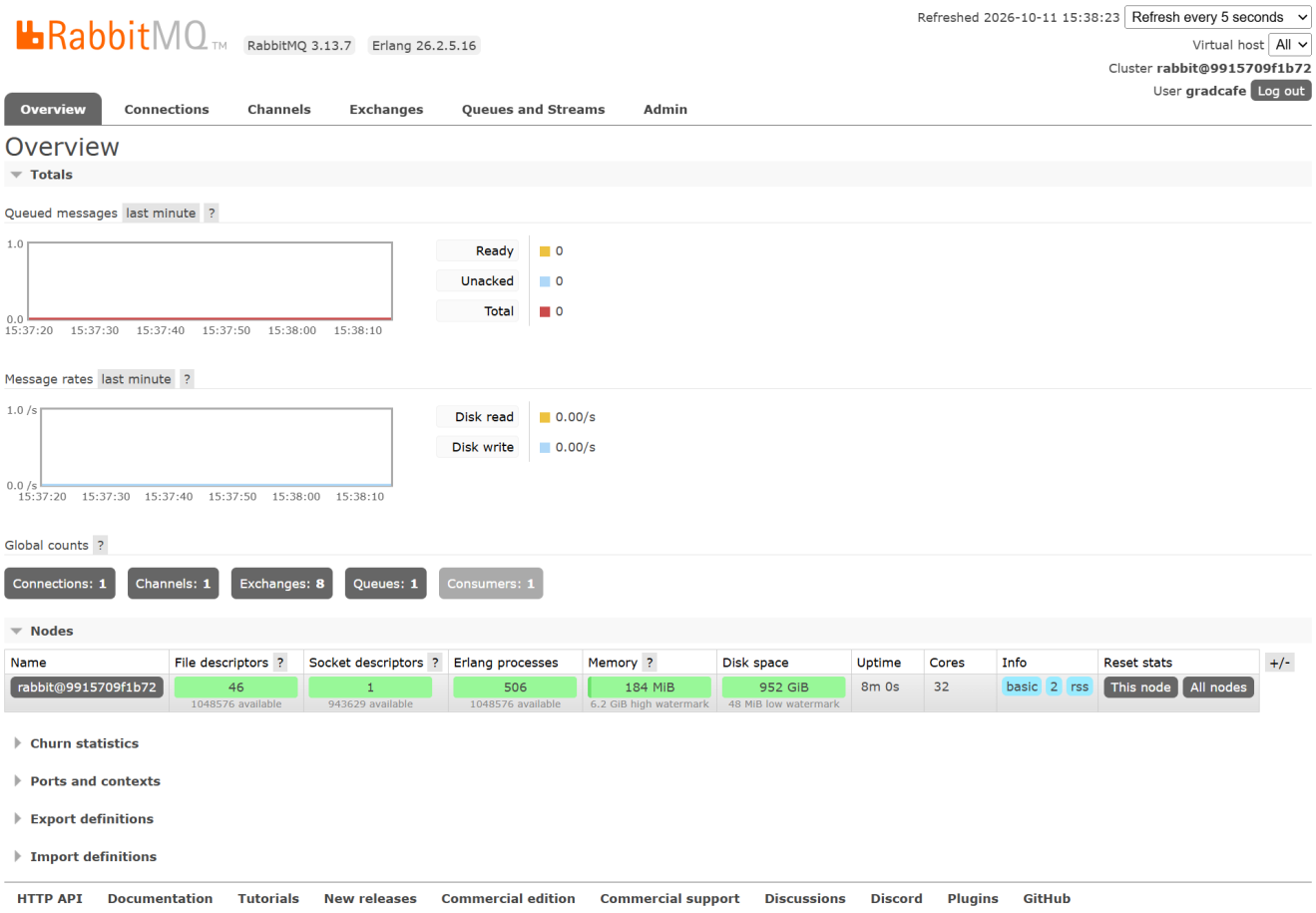
Current screenshots

Applicant dashboard



Current database-backed dashboard and task actions from the running web service.

RabbitMQ management overview



Authenticated management overview; queue is idle at capture time. No password is included.